

MPI 课程总结

Cours 2: Introduction a MPI et communications point-a-point

2.1 MPI 模型

- **MPI = Message Passing Interface** 定义：MPI est une interface standard de communication entre processus pour les programmes parallèles à mémoire distribuée, implémentée par des bibliothèques comme MPICH ou Open MPI. / MPI 是一种用于分布式内存并行程序的进程间通信标准接口，由 MPICH 或 Open MPI 等库实现。
- Exemple simple : envoyer un message d'un processus à un autre. / 简单示例：从一个进程向另一个进程发送一条消息。
- **MPI process**：Un processus MPI est une instance d'exécution d'un programme participant à une application parallèle MPI. Chaque processus possède son propre espace mémoire (adressage privé), son propre rang dans un communicateur, et communique avec les autres processus par des appels MPI explicites. / MPI 进程是参与 MPI 并行应用的程序执行实例。每个进程都有自己的内存空间（私有寻址）、在通信子中的唯一标识符，以及通过显式的 MPI 调用与其他进程通信。
- **communicator** 定义：En MPI, un communicateur est un objet représentant un groupe de processus pouvant communiquer entre eux de manière isolée. Il sert à organiser les communications dans une application parallèle. / 在 MPI 中，communicator 是一个对象，表示一组可以相互通信的进程。它用于组织并行应用中的通信。

Exemple : MPI_Comm_split sépare MPI_COMM_WORLD en deux communicateurs selon la parité du rang global. Dans chaque nouveau communicateur, les rangs sont redéfinis à partir de 0 ; ainsi, le processus de rang local 0 peut communiquer avec celui de rang local 1 à l'intérieur de son propre communicateur. / MPI_Comm_split 按全局 rank 的奇偶性把 MPI_COMM_WORLD 分成两个通信子。在每个新通信子中，rank 会从 0 重新编号；因此，本地 rank 0 的进程可以在各自通信子内部与本地 rank 1 的进程通信。

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char *argv[]) {
    // initialise l'environnement d'exécution MPI / 初始化 MPI 环境: `MPI_Init` 用于初始化 MPI 环境。
    MPI_Init(&argc, &argv);
    // 获取全局 communicator 的秩与大小: `MPI_Comm_rank` 和 `MPI_Comm_size` 给出 `MPI_COMM_WORLD` 的信息。
    int world_rank, world_size;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
    // Sépare MPI_COMM_WORLD en deux communicateurs : rangs pairs et rangs impairs
    MPI_Comm new_comm;
    MPI_Comm_split(MPI_COMM_WORLD, world_rank % 2, world_rank, &new_comm);
    int new_rank, new_size;
    MPI_Comm_rank(new_comm, &new_rank); //donne le rang du processus courant dans le communicateur
    MPI_Comm_size(new_comm, &new_size); //donne le nombre total de processus dans le communicateur
    printf("Processus global %d sur %d - Processus local %d sur %d\n",
        world_rank, world_size, new_rank, new_size);
    if (new_rank == 0) {
        int message = 42;
        MPI_Send(&message, 1, MPI_INT, 1, 0, new_comm);
        printf("Processus %d a envoyé le message %d\n", new_rank, message);
    } else if (new_rank == 1) {
        int message;
        MPI_Recv(&message, 1, MPI_INT, 0, 0, new_comm, MPI_STATUS_IGNORE);
        printf("Processus %d a reçu le message %d\n", new_rank, message);
    }
    MPI_Comm_free(&new_comm); // Libère le nouveau communicateur / 释放新通信子
    MPI_Finalize(); // termine l'environnement MPI / 终止 MPI 环境。
    return 0;
}
```

- **MPI_COMM_WORLD**：communicateur par défaut contenant tous les processus MPI lancés au départ. / 默认通信子；`mpirun -np P ./a.out` 启动出的全部 P 个进程默认都在这里。
- **rank**：identifiant d'un processus dans un communicateur donné, compris entre 0 et size-1. / 进程在某个通信子中的编号；范围 0..size-1；只对当前通信子有意义。
- **size**：nombre total de processus appartenant à ce communicateur. / 通信子中的进程总数；不是 CPU 核数，而是该通信子包含的 MPI 进程数。

2.2 MPI_Send

- `MPI_Send` retourne lorsque l'envoi est localement termine : soit les donnees de `buf` ont ete copiees dans un buffer interne MPI, soit le transfert direct a pu progresser apres la synchronisation/mise en correspondance necessaire avec le recepteur ; dans tous les cas, le tampon `buf` peut alors etre reutilise en toute securite. / `MPI_Send` 通常要么先把 `buf` 中的数据复制到 MPI 的内部缓冲区, 要么先与接收方完成必要的匹配/同步后直接传输数据, 并在发送方一侧的发送已完成、`buf` 可安全复用时返回。

```
int MPI_Send(
    void *buf,           // 待发送缓冲区首地址
    int count,           // 发送元素个数, 不是字节数
    MPI_Datatype datatype, // 每个元素按什么 MPI 类型解释
    int dest,            // 目标进程 rank
    int tag,             // 消息标签
    MPI_Comm comm        // 通信子
);
```

2.3 MPI_Recv

- `MPI_Recv` retourne uniquement lorsque le message correspondant a ete recu et que les donnees ont effectivement ete placees dans le tampon de reception `buf`, apres la mise en correspondance necessaire avec l'emetteur. / `MPI_Recv` 将数据接收到接收缓冲区 `buf` 中, 必要时先与发送方完成匹配, 并且只有当数据已经实际放入 `buf` 后才返回。
- 若接收端 `count` 小于发送端 `count`, 会发生截断错误。
- 若`buf` 太小, 会越界访问可能 `seg fault`。

```
int MPI_Recv(
    void *buf,           // 接收缓冲区首地址, 必须提前分配
    int count,           // 最多可接收元素个数上限, 不是必须收到多少个
    MPI_Datatype datatype, // 按什么类型解释收到的数据
    int source,          // 指定发送者, 也可写 MPI_ANY_SOURCE
    int tag,             // 指定标签, 也可写 MPI_ANY_TAG
    MPI_Comm comm,       // 通信子
    MPI_Status *status    // 返回实际收到的 source/tag/error
);
```

2.4 MPI_Status 与 MPI_Get_count

```
typedef struct {
    int MPI_SOURCE; // 实际发送者 rank
    int MPI_TAG;    // 实际消息标签
    int MPI_ERROR;  // 错误码
} MPI_Status;
```

```
int MPI_Get_count(
    MPI_Status *status, // MPI_Recv 返回的状态
    MPI_Datatype datatype, // 按什么类型统计元素个数
    int *count          // 输出: 实际收到多少个 datatype 元素
);
```

- `MPI_Status` permet de recuperer apres une reception des informations sur le message effectivement recu, notamment la vraie source et le vrai tag ; il est particulierement utile avec `MPI_ANY_SOURCE` ou `MPI_ANY_TAG` . / `MPI_Status` 可以在一次接收之后获取实际收到的消息信息, 尤其是真实的发送源和真实的标签; 它在使用 `MPI_ANY_SOURCE` 或 `MPI_ANY_TAG` 时特别有用。
- `MPI_Get_count` permet de connaitre le nombre reel d'elements recus d'un type donne ; il est particulierement utile pour les messages de taille variable ou lorsque la taille exacte du message n'etait pas connue a l'avance. / `MPI_Get_count` 可以知道实际收到的某种给定类型元素的真实个数; 它对变长消息, 或消息精确长度事先未知的情况特别有用。

2.5 MPI 基础类型

C 类型	char	int	float	double	long double	字节流
MPI 类型	MPI_CHAR	MPI_INT	MPI_FLOAT	MPI_DOUBLE	MPI_LONG_DOUBLE	MPI_BYTE

2.6 communications point à point

定义：Les communications point à point sont des communications effectuées entre deux processus : un processus émetteur et un processus récepteur. / 点对点通信是在两个进程之间进行的通信：一个发送进程和一个接收进程。

Cours 3: Communications point-a-point avancees

3.1 阻塞通信的精确定义

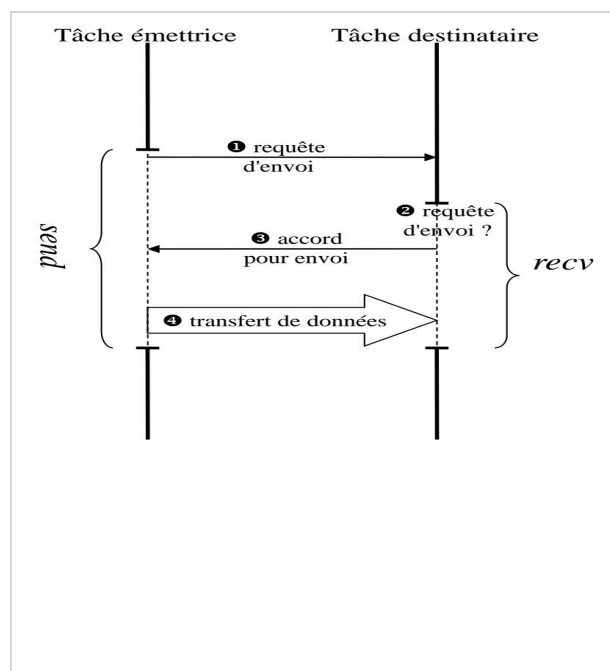
定义：Dans une communication bloquante, l'appel de la fonction ne se termine que lorsque l'opération de communication est terminée. Le processus appelant reste bloqué jusqu'à ce moment-là. / 在阻塞通信中，函数调用只有在通信操作完成时才会结束。调用进程在此之前会一直阻塞。

- Un envoi bloquant garantit seulement l'achèvement local : à son retour, le tampon d'envoi peut être réutilisé, sans garantir que le message a déjà été traité par le récepteur. / 阻塞发送只保证本地完成：返回时，发送缓冲区可以重用，但不保证接收方已经处理完消息。
- Une réception bloquante retourne lorsque les données ont été effectivement placées dans le tampon de réception ; celui-ci peut donc être lu immédiatement. / 阻塞接收返回时，数据已经实际放入接收缓冲区；可以立即读取。

```
MPI_Send(buffer, count, datatype, dest, tag, comm);  
MPI_Recv(buffer, count, datatype, source, tag, comm, status);
```

3.2 三种阻塞发送模式

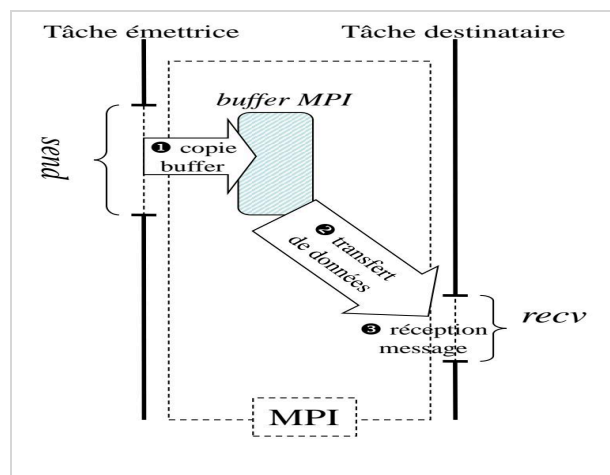
3.2.1 MPI_Ssend 同步发送



```
int MPI_Ssend(  
    void *buf,           // 待发送缓冲区首地址  
    int count,           // 元素个数  
    MPI_Datatype datatype, // 元素类型  
    int dest,            // 目标 rank  
    int tag,             // 消息标签  
    MPI_Comm comm        // 通信子  
);
```

- **synchrone** : l'envoi ne se termine qu'après la mise en correspondance avec la reception associee. / 只有在与对应接收建立匹配后，发送才会完成。
- **Avantage** : ce mode est utile lorsque l'emetteur et le recepteur sont bien coordonnes ; il evite de compter sur un buffering pour permettre un retour anticipe de l'appel. / 适合发送方和接收方配合较紧的场景；它不是靠缓冲来让发送提前返回。
- **Inconvenient** : si le recepteur n'est pas pret, l'emetteur peut attendre inutilement plus longtemps. / 如果接收方没有及时准备好，发送方可能要额外等待。

3.2.2 MPI_Bsend 缓冲发送



```
int MPI_Bsend(  
    void *buf,           // 用户发送缓冲区  
    int count,           // 元素个数  
    MPI_Datatype datatype, // 元素类型  
    int dest,            // 目标 rank  
    int tag,             // 消息标签  
    MPI_Comm comm        // 通信子  
);
```

- **bufferise** : l'appel retourne lorsque les donnees ont ete copiees dans le buffer attache par l'utilisateur a MPI. / 当数据已经复制到用户为 MPI 附加的缓冲区后，调用返回。
- **Avantage** : l'envoi peut se terminer meme si la reception correspondante n'a pas encore ete postee ; cela aide a decoupler l'emetteur et le recepteur.

/ 即使接收方还没发出对应接收，发送也可能先完成；
这样就可以把发送方和接收方解耦。

- **Inconvenient** : il y a une copie supplémentaire, donc un surcote en memoire et en temps, et la capacite du buffer reste limitee. / 会有额外拷贝，因此增加时间和内存开销，而且缓冲区容量有限。

3.2.3 手动分配缓冲区

用户可以使用自己在进程地址空间中分配的缓冲区来替代 MPI 默认使用的缓冲区。

```
int MPI_Buffer_attach(void *buf, int sz); //attache un buffer utilisateur pour MPI_Bsend
int MPI_Buffer_detach(void **buf_addr, int *sz); //detache le buffer et recupere son adresse/sa taille
```

```
#define BUFFSIZE 100000 // 100 KB
int sz; // 实际附加的缓冲区大小
char *mpi_buf; // 用户分配、附加给 MPI_Bsend 的缓冲区
mpi_buf = malloc(BUFFSIZE);
MPI_Buffer_attach(mpi_buf, BUFFSIZE); // 同时只能有一个这样的缓冲区
MPI_Bsend(msg1, ...); // 发送第一条消息
MPI_Bsend(msg2, ...); // 发送第二条消息
MPI_Buffer_detach((void **)&mpi_buf, &sz);
free(mpi_buf);
```

- **attach** 的是供 **MPI_Bsend** 使用的附加 MPI 缓冲区，不是应用自己的发送数据缓冲区。
- 接收端通常不需要附加缓冲区。
- 只有确认没有未完成的 **MPI_Bsend** 还在使用它时，才能 **detach/free**。

3.2.4 MPI_Send 标准发送

- **standard** : 不固定采用哪种协议，由 MPI 实现自己选。
- 常见行为：小消息更像 **buffered/eager**，大消息更像 **synchronous/rendezvous**。
- 所以：同一段代码可能小消息能跑，大消息死锁。
- 诊断方法：把 **MPI_Send** 临时全部换成 **MPI_Ssend**；若卡住，说明通信顺序有 bug。

3.3 非阻塞通信的本质

- 定义：Dans une communication non bloquante, l'appel retourne sans attendre la fin de l'opération : la communication est seulement initiée, et son achèvement doit ensuite être vérifié ou attendu explicitement avec **MPI_Wait*** ou **MPI_Test***. / 在非阻塞通信中，函数调用不会等待操作完成就返回：通信只是被启动，之后必须用 **MPI_Wait*** 或 **MPI_Test*** 显式检查或等待其完成。

L'opération se fait en arrière-plan et doit être finalisée explicitement. / 通信操作在后台进行，并且需要显式完成。

- Après **MPI_Isend**, le tampon d'envoi ne peut pas être modifié immédiatement. / 在 **MPI_Isend** 之后，发送缓冲区不能立即修改。
- Après **MPI_Irecv**, le tampon de réception ne peut pas être lu immédiatement. / 在 **MPI_Irecv** 之后，接收缓冲区不能立即读取。

```
MPI_Isend(buffer, count, datatype, dest, tag, comm, request);
MPI_Irecv(buffer, count, datatype, source, tag, comm, request);
```

优点：Les communications non bloquantes permettent de ne pas bloquer le processus pendant l'échange de données. Elles offrent ainsi la possibilité de recouvrir communication et calcul, ce qui peut réduire le temps d'attente et améliorer les performances globales. / 非阻塞通信不会让进程在数据交换期间一直阻塞。这样就有可能把通信和计算重叠起来，从而减少等待时间并改善整体性能。

3.4 MPI_Request 与非阻塞发送/接收

```
int MPI_Isend(
    void *buf, // 发送缓冲区；通信只是被启动了，但在函数返回时它不一定已经完成；完成前不可改写
    int count, // 元素个数
    MPI_Datatype datatype, // 元素类型
    int dest, // 目标 rank
    int tag, // 消息标签
    MPI_Comm comm, // 通信子
    MPI_Request *req // 输出：本次通信请求的标识，之后通常要配合MPI_Wait或MPI_Test 判断发送是否完成。
);
```

```

int MPI_Irecv(
    void *buf,                // 接收缓冲区；通信只是被启动了，但在函数返回时它不一定已经完成；完成前不可读取
    int count,                // 最多接收多少个元素
    MPI_Datatype datatype,    // 元素类型
    int source,               // 指定发送者或 MPI_ANY_SOURCE
    int tag,                  // 指定标签或 MPI_ANY_TAG
    MPI_Comm comm,            // 通信子
    MPI_Request *req          // 输出：本次通信请求的标识，之后通常要配合MPI_Wait或MPI_Test判断接收是否完成。
);

```

3.5 完成请求： MPI_Wait / MPI_Test / MPI_Waitall

MPI_Wait bloque jusqu'a ce que la communication non bloquante associee a ***req** soit terminee. Pour un envoi, cela signifie que le tampon d'envoi peut de nouveau etre reutilise en toute securite ; pour une reception, cela signifie que les donnees ont effectivement ete placees dans le tampon de reception. / **MPI_Wait** 会一直阻塞，直到 ***req** 对应的非阻塞通信完成时才返回；对于发送，这表示发送缓冲区已可安全复用；对于接收，这表示数据已实际写入接收缓冲区。

```

int MPI_Wait(
    MPI_Request *req,         // 等待该请求完成；返回后通常变成 MPI_REQUEST_NULL 表示请求已失效
    MPI_Status *sta          // 返回状态；可用 MPI_STATUS_IGNORE
);

```

```

MPI_Request req;
MPI_Status sta;
MPI_Isend(buf, N, MPI_BYTE, dest, tag1, comm, &req);
instruction1; // 这里的命令不能改写 buf，而且读取的数据也不一定是最新的，因为发送可能还没完成。
instruction2; //但是可以执行与 buf 无关的其他计算。
...
instructionN;
MPI_Wait(&req, &sta); // 这里如果发送还没完成，会一直等；如果已经完成了，就直接返回。
instructionN+1; // 这里可以安全地改写 buf 了。

```

MPI_Test ne bloque pas ; il verifie immediatement si la communication non bloquante associee a ***req** est deja terminee. Si oui, ***flag != 0** et l'operation a alors la meme semantique d'achevement qu'apres un **MPI_Wait** ; sinon, l'appel retourne tout de suite avec ***flag == 0** . / **MPI_Test** 不会一直阻塞；它立即检查 ***req** 对应的非阻塞通信是否已经完成。若已完成，则 ***flag != 0**，并且这次通信达到与 **MPI_Wait** 返回时相同的完成语义；若未完成，则立刻返回，***flag == 0**。

```

int MPI_Test(
    MPI_Request *req,         // 测试该请求是否完成；若已完成，通常会变成 MPI_REQUEST_NULL
    int *flag,                // 输出完成标志：完成则非 0，否则为 0
    MPI_Status *sta          // 若完成则返回状态；否则其内容不应依赖
);

```

```

MPI_Request req;
MPI_Status sta;
int flag = 0;

MPI_Irecv(msg, N, MPI_BYTE, dest, tag, comm, &req); // 先挂一个非阻塞接收
do {
    instruction1; // 这里可以继续做别的计算
    ...
    instructionN;
    MPI_Test(&req, &flag, &sta); // 只检查是否完成，不会一直阻塞
} while (!flag); // 跳出循环时，msg 中的数据才真正可读

```

MPI_Waitall bloque jusqu'a ce que toutes les communications non bloquantes du tableau de requetes soient terminees. Pour les envois, cela signifie que tous les tampons d'envoi correspondants peuvent etre reutilises ; pour les receptions, cela signifie que les donnees ont ete effectivement placees dans leurs tampons de reception respectifs. / **MPI_Waitall** 会一直阻塞，直到给定请求数组中的所有非阻塞通信都完成时才返回；对其中的发送，这表示对应发送缓冲区都已可安全复用；对其中的接收，这表示对应数据都已实际写入各自的接收缓冲区。

```

int MPI_Waitall(
    int nb_req,               // 请求个数
    MPI_Request *tab_req,     // 请求数组；其中每个元素对应一条未完成通信
    MPI_Status *tab_sta       // 状态数组；每个请求完成后的状态写在对应位置
);

```

- Autres fonctions utiles sur des ensembles de requetes : `MPI_Testall` teste si toutes les requetes sont terminees ; `MPI_Waitany` / `MPI_Testany` attendent/testent jusqu'a la terminaison d'une requete et renvoient son indice ; `MPI_Waitsome` / `MPI_Testsome` attendent/testent jusqu'a la terminaison d'une ou plusieurs requetes et renvoient les indices correspondants. / 其他常用的多请求函数： `MPI_Testall` 用于测试一组请求是否全部完成； `MPI_Waitany` / `MPI_Testany` 会等待/测试直到某一个请求完成，并返回它的下标； `MPI_Waitsome` / `MPI_Testsome` 会等待/测试直到一个或多个请求完成，并返回对应下标。

```
MPI_Request req[4];
MPI_Status sta[4];
gauche = (rang + P - 1) % P;
droite = (rang + 1) % P;
MPI_Isend(&x[1], 1, MPI_DOUBLE, gauche, tag, comm, req); // 向左邻居发送边界值
MPI_Isend(&x[N], 1, MPI_DOUBLE, droite, tag, comm, req + 1); // 向右邻居发送边界值
MPI_Irecv(&x[0], 1, MPI_DOUBLE, gauche, tag, comm, req + 2); // 从左邻居接收 halo
MPI_Irecv(&x[N + 1], 1, MPI_DOUBLE, droite, tag, comm, req + 3); // 从右邻居接收 halo
MPI_Waitall(4, req, sta); // 四条通信全部完成后，x[0] 和 x[N+1] 才可安全使用
```

3.6 阻塞与非阻塞可以混搭

- `MPI_Isend` 可以配 `MPI_Recv`。
- `MPI_Send` 可以配 `MPI_Irecv`。
- `MPI_Isend` 也可以配 `MPI_Irecv`。

匹配是否成立看的是 (`comm`, `source`, `tag`, `datatype/count` 的语义一致性) 而不是看两边是否都“同为阻塞”或“同为非阻塞”。

3.7 调用类型 vs 发送模式

调用类型 \ 发送模式	Standard	Bufferise	Synchrone
Bloquant	<code>MPI_Send</code>	<code>MPI_Bsend</code>	<code>MPI_Ssend</code>
Non bloquant	<code>MPI_Isend</code>	<code>MPI_Ibsend</code>	<code>MPI_Issend</code>

- `bloquant/non bloquant`：函数什么时候返回。
- `standard/bufferise/synchrone`：发送是按什么语义完成的。
- `asynchrone` 不是这里的正式分类；非阻塞调用也不自动保证一定有明显后台推进。

3.8 任意大小消息： `MPI_Iprobe` / `MPI_Probe`

Le probleme avec `MPI_Recv` est qu'il faut fournir a l'avance une borne superieure pour `count` ; cela n'est pas pratique pour les messages de taille variable. / `MPI_Recv` 的问题：必须事先给出 `count` 上界；对变长消息不方便。

`MPI_Iprobe` ne bloque pas ; il verifie seulement si un message correspondant a (`source`, `tag`, `comm`) est deja arrive. / `MPI_Iprobe` 不会阻塞；它只检查当前是否已经有一条匹配 (`source`, `tag`, `comm`) 的消息到达。

```
int MPI_Iprobe(
    int source,           // 指定发送者或 MPI_ANY_SOURCE
    int tag,              // 指定标签或 MPI_ANY_TAG
    MPI_Comm comm,        // 通信子
    int *flag,             // 是否已有匹配消息到达；到达则非 0，否则为 0
    MPI_Status *sta        // 若到达把该消息的状态信息写入 `*sta`
);
```

`MPI_Probe` bloque jusqu'a l'arrivee d'un message correspondant a (`source`, `tag`, `comm`) ; au retour, `*sta` contient deja les informations sur ce message, mais le message lui-meme n'a pas encore ete consomme et il faut ensuite appeler `MPI_Recv` pour le recevoir effectivement. / `MPI_Probe` 会阻塞，直到有一条匹配 (`source`, `tag`, `comm`) 的消息到达；返回时 `*sta` 已包含该消息信息，但消息本身仍未被消费，后续还必须再调用一次 `MPI_Recv`。

```
int MPI_Probe(
    int source,           // 指定发送者或 MPI_ANY_SOURCE
    int tag,              // 指定标签或 MPI_ANY_TAG
    MPI_Comm comm,        // 通信子
    MPI_Status *sta        // 返回该消息状态
);
```

标准流程： `Probe / Iprobe` 拿到 `status` -> `MPI_Get_count` 得到真实长度 -> `malloc` 合适大小的缓冲区 -> `MPI_Recv` 按 `status.MPI_SOURCE` 和 `status.MPI_TAG` 真正接收。

```
MPI_Status sta;
int taille, arrive;
do {
    instruction1; // 先做别的计算
    ...
    instructionN;
    MPI_Iprobe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &arrive, &sta); // 轮询是否已有任意匹配消息到达
} while (!arrive); // 跳出时说明 sta 里已经有该消息的元信息
MPI_Get_count(&sta, MPI_BYTE, &taille); // 查询该消息的真实字节数
char *buf = malloc(taille); // 按真实大小分配接收缓冲区
MPI_Recv(buf, taille, MPI_BYTE, sta.MPI_SOURCE, sta.MPI_TAG, MPI_COMM_WORLD, &sta); // 取走消息
```

Cours 4: Communications collectives

4.1 集合通信的共同约束

定义：Les communications collectives sont des opérations de communication impliquant tous les processus d'un même communicateur, qui participent ensemble à une même opération MPI. / 集合通信是涉及同一通信子中所有进程的通信操作，这些进程共同参与同一个 MPI 操作。

- 所有进程必须共同调用同一个 MPI 集合通信操作，并且算法顺序必须兼容。
- 不能把 collective 放进可能在不同进程上分叉的条件分支里。
- collective 通常可以用 `send/recv` 手工实现，但 MPI 自带版本通常更优化。
- `MPI_All*` 一般没有显式 `root`；所有进程都参与，所有进程都拿到结果。
- collective 会被最慢的那个进程拖住，所以负载不均衡会直接放大通信成本。
- Dans une rooted collective, tous les processus doivent appeler la meme operation, mais tous les parametres n'ont pas necessairement une signification sur tous les processus. / 在带根的集合通信里，所有进程都要调用同一个操作，但不是所有参数在所有进程上都有意义。
- En general, `sendbuf` et `recvbuf` ne doivent pas se chevaucher arbitrairement ; un traitement en place n'est permis que dans les cas explicitement autorises par MPI avec `MPI_IN_PLACE`. / 一般情况下，`sendbuf` 和 `recvbuf` 不应随意重叠；若要原地操作，必须是 MPI 明确允许的 `MPI_IN_PLACE` 用法。

4.2 MPI_Barrier

`MPI_Barrier` est une primitive de synchronisation pure : elle ne transfere aucune donnee et bloque le processus appelant jusqu'a ce que tous les processus de `comm` aient atteint cette barriere. / `MPI_Barrier` 是纯同步原语，不传输任何数据；它会阻塞当前进程，直到 `comm` 中的所有进程都进入这次屏障。

```
int MPI_Barrier( MPI_Comm comm); // 要同步的通信子
```

```
MPI_Init(&argc, &argv);
travail_phase_1(); // 各进程先做自己的第一阶段计算
MPI_Barrier(MPI_COMM_WORLD); // 所有进程都到这里后，屏障才会放行
travail_phase_2(); // 然后大家再进入第二阶段
MPI_Finalize();
```

- Le retour d'un processus hors de la barriere signifie seulement que tous les processus ont atteint cette barriere. / 某个进程从 `Barrier` 返回，只能说明所有进程都已进入屏障。
- Cela ne garantit pas que tous les autres processus en sont deja sortis. / 不保证其他进程已经从这次屏障中返回。
- Il s'agit d'une synchronisation pure, et non d'une synchronisation accompagnee d'un echange de donnees. / 这是“纯同步”，不是“同步 + 数据交换”。

Comment MPI gère-t-il la synchronisation entre les processus ? Donnez un exemple de synchronisation explicite et implicite. MPI 如何处理进程之间的同步？请分别给出显式同步和隐式同步的例子

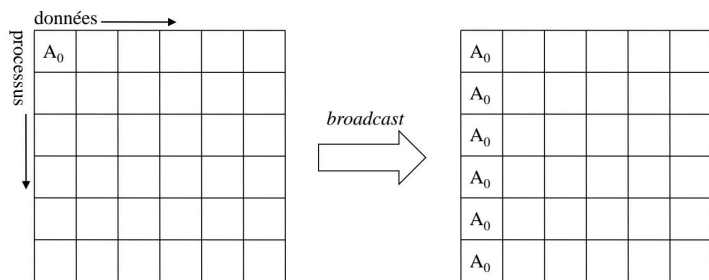
MPI utilise des barrières (`MPI_Barrier`) pour la synchronisation explicite. / MPI 使用屏障 (`MPI_Barrier`) 进行显式同步。

Les communications collectives comme `MPI_Bcast` impliquent une synchronisation implicite. De même, certaines communications point à point synchrones, comme `MPI_Ssend`, introduisent aussi une synchronisation implicite entre les processus. / 像 `MPI_Bcast` 这样的集合通信包含隐式同步。同样，某些同步的点点对通信，如 `MPI_Ssend`，也会在进程之间引入隐式同步。

4.3 MPI_Bcast

MPI_Bcast diffuse une meme donnee detenue initialement par le processus racine **root** vers tous les processus du communicateur. /

MPI_Bcast 负责把根进程 **root** 持有的一份数据复制到通信子中的所有进程。



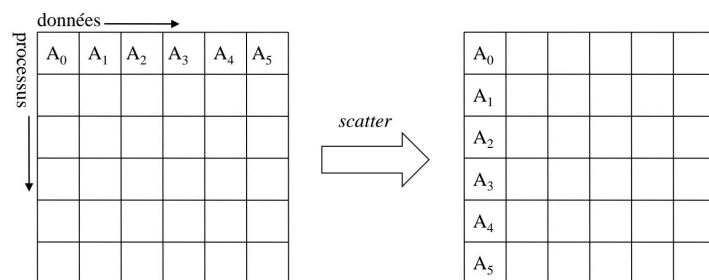
```
int MPI_Bcast(
    void *buf,           // root 上是待广播数据；其他进程上是接收缓冲区，接收缓冲区必须已经分配好。
    int count,           // 广播多少个 datatype 元素
    MPI_Datatype datatype, // 元素类型
    int root,            // 根进程 rank；所有进程必须传同一个 root
    MPI_Comm comm        // 通信子
);
```

```
int main(int argc, char *argv[]) {
    int rank, size, data;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (rank == 0) {data = 42;} // 只有根进程最初持有有效数据
    MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
    // `MPI_Bcast` diffuse la valeur de `data` du processus 0 vers tous les autres processus.
    // `MPI_Bcast` 会把进程 0 中 `data` 的值广播给所有其他进程。
    printf("Process %d received data: %d\n", rank, data);
    MPI_Finalize();
    return 0;
}
```

4.4 MPI_Scatter

MPI_Scatter permet au processus racine de decouper un tableau global en blocs de taille fixe et d'en distribuer un a chaque processus. /

MPI_Scatter 由根进程把全局数组按固定块大小拆开，每个进程接收其中一块。



```
int MPI_Scatter(
    void *sendbuf,        // 根进程上的全局发送数组；非根进程通常可忽略
    int sendcount,        // 发给每个进程多少个 sendtype 元素
    MPI_Datatype sendtype, // 发送元素类型
    void *recvbuf,        // 每个进程本地接收缓冲区
    int recvcnt,          // 本地接收多少个 recvtype 元素
    MPI_Datatype recvtype, // 接收元素类型
    int root,             // 根进程 rank
    MPI_Comm comm         // 通信子
);
```

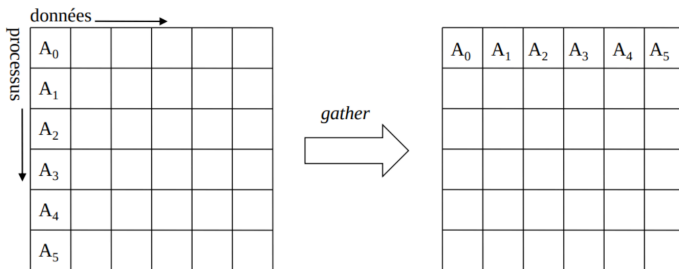
Principe : **root** decoupe **sendbuf** en **P** blocs de meme taille suivant l'ordre des rangs, puis envoie le bloc **p** au processus **p**. / 机制: **root** 把 **sendbuf** 按 rank 顺序切成 **P** 个等长块，第 **p** 块发给进程 **p**。


```
int moi, P, root = 0, val_recue;
int *sbuf = NULL;
MPI_Comm_rank(MPI_COMM_WORLD, &moi);
MPI_Comm_size(MPI_COMM_WORLD, &P);
if (moi == root) {
    sbuf = malloc(P * sizeof(int));
    for (int p = 0; p < P; p++) sbuf[p] = 100 + p; // sbuf[p] -> rank p
}
MPI_Scatter(sbuf, 1, MPI_INT, &val_recue, 1, MPI_INT, root, MPI_COMM_WORLD);
printf("P%d: val_recue = %d\n", moi, val_recue);
```

- **Contrainte 1** : dans la forme simple, on a en general `recvcount == sendcount` , et les types doivent etre compatibles. / 约束 1: 简单形式下通常 `recvcount == sendcount` , 类型也应匹配。
- **Contrainte 2** : sur `root` , `sendbuf` doit contenir au moins `P * sendcount` elements. / 约束 2: `root` 上的 `sendbuf` 至少要有 `P * sendcount` 个元素。
- **Contrainte 3** : le bloc envoye au rang `p` commence a l'adresse `sendbuf + p * sendcount` ; `root` recoit aussi son propre bloc. / 约束 3: 发给 rank `p` 的块从 `sendbuf + p * sendcount` 开始; `root` 自己也会收到自己的块。
- **Contrainte 4** : tous les processus doivent fournir le meme `root` et le meme `comm` ; si les tailles de blocs different, il faut utiliser `MPI_Scatterv` . / 约束 4: 所有进程必须用同一个 `root` / `comm` ; 块大小不同就改用 `MPI_Scatterv` 。

4.5 MPI_Gather / MPI_Allgather

`MPI_Gather` est l'operation inverse de `Scatter` : chaque processus fournit un bloc local, et le processus racine les rassemble dans un tableau suivant l'ordre des rangs. / `MPI_Gather` 是 `Scatter` 的逆操作: 每个进程提供本地一块数据, 由根进程按 rank 顺序收集到一个数组里。



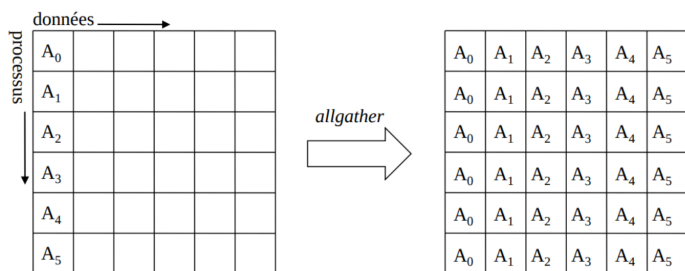
```
int MPI_Gather(
    void *sendbuf,           // 每个进程本地要发送给根进程的数据
    int sendcount,           // 本地发送元素个数
    MPI_Datatype sendtype,   // 发送元素类型
    void *recvbuf,           // 根进程上的接收数组; 非根进程通常可忽略
    int recvcount,           // 从每个进程收多少个元素, 一般等于 sendcount
    MPI_Datatype recvtype,   // 接收元素类型
    int root,                // 根进程 rank
    MPI_Comm comm            // 通信子
);
```

```
int moi, P, root = 0, val_calcul;
int *rcvbuf = NULL;
MPI_Comm_rank(MPI_COMM_WORLD, &moi);
MPI_Comm_size(MPI_COMM_WORLD, &P);
val_calcul = 10 * moi; // 每个进程先得到自己的局部结果
if (moi == root) rcvbuf = malloc(P * sizeof(int)); // 只有 root 需要接收完整结果数组
MPI_Gather(&val_calcul, 1, MPI_INT, rcvbuf, 1, MPI_INT, root, MPI_COMM_WORLD);
if (moi == root) {
    for (int p = 0; p < P; p++) {
        printf("rcvbuf[%d] = %d\n", p, rcvbuf[p]); // rcvbuf[p] 对应 rank p 发来的值
    }
}
```

- S'il y a `P` processus, `rcvbuf` sur le processus racine doit pouvoir contenir au moins `P * recvcount` elements. / 若有 `P` 个进程, 则根进程上的 `rcvbuf` 至少要能装 `P * recvcount` 个元素。
- Le bloc provenant du processus `p` est ecrit a partir de `rcvbuf + p * recvcount` sur la racine. / 来自进程 `p` 的块写到根进程 `rcvbuf + p * recvcount` 开始的位置。

`MPI_Allgather` organise les donnees comme `MPI_Gather` , mais le resultat final est recopie sur tous les processus et non pas seulement sur la racine. / `MPI_Allgather` 与 `MPI_Gather` 的数据组织相同, 只是结果不只留在根进程, 而是复制给所有进程。

- **Cout memoire** : chaque processus doit stocker la totalite du resultat global dans son propre `recvbuf` , ce qui peut devenir couteux si ce resultat est volumineux. / **内存代价** : 每个进程都必须在自己的 `recvbuf` 中保存完整的全局结果；如果结果很大，这会带来明显的内存开销。



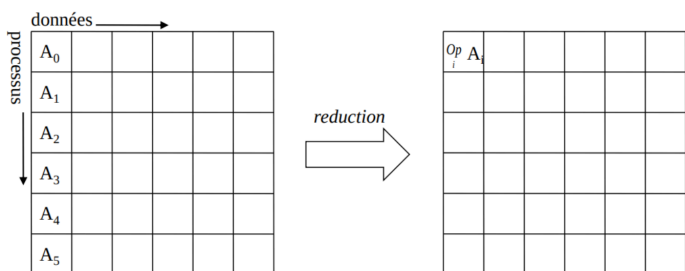
```
int MPI_Allgather(
    void *sendbuf,           // 每个进程本地发送块
    int sendcount,           // 本地发送元素个数
    MPI_Datatype sendtype,   // 发送元素类型
    void *recvbuf,           // 每个进程都要拥有的全局结果数组
    int recvcnt,             // 从每个进程接收多少个元素
    MPI_Datatype recvttype,  // 接收元素类型
    MPI_Comm comm            // 通信子
);
```

4.5 bis rooted collectives 里谁的 buffer 有效

- **MPI_Bcast** : seul `root` possede une valeur initiale significative avant l'appel ; apres l'appel, tous les processus ont le meme contenu dans `buf` . / **MPI_Bcast** : 只有 `root` 在调用前持有有效初值；调用后所有进程的 `buf` 都持有相同结果。
- **MPI_Scatter** : seul le `sendbuf` de `root` a une signification pour l'envoi global ; le `recvbuf` local de chaque processus contient son bloc reçu. / **MPI_Scatter** : 只有 `root` 的 `sendbuf` 在发送意义上真正有用；每个进程自己的 `recvbuf` 都有效。
- **MPI_Gather** : le `sendbuf` de chaque processus est significatif, mais seul le `recvbuf` de `root` contient le resultat complet. / **MPI_Gather** : 每个进程的 `sendbuf` 都有效，但只有 `root` 的 `recvbuf` 真正保存完整结果。
- **MPI_Reduce** : le `sendbuf` de chaque processus participe a la reduction, mais seul le `recvbuf` de `root` contient le resultat final. / **MPI_Reduce** : 每个进程的 `sendbuf` 都参与归约，但只有 `root` 的 `recvbuf` 上结果有效。
- **MPI_Allgather** / **MPI_Allreduce** : il n'y a pas de `root` et tous les processus recoivent le resultat final. / **MPI_Allgather** / **MPI_Allreduce** : 没有 `root` ；所有进程最终都得到结果。

4.6 MPI_Reduce

MPI_Reduce combine les valeurs locales de tous les processus a l'aide d'un operateur de reduction `op` , puis ecrit le resultat final uniquement sur le processus racine. / **MPI_Reduce** 会把每个进程的局部值按某个归约算子 `op` 合并，最终结果只写到根进程。



```
int MPI_Reduce(
    void *sendbuf,           // 每个进程本地输入数据
    void *recvbuf,           // 根进程上的输出缓冲区
    int count,               // 每个进程参与归约的元素个数
    MPI_Datatype datatype,   // 元素类型
    MPI_Op op,               // 归约算子, 如 MPI_SUM / MPI_MAX / MPI_MIN
    int root,                // 根进程 rank
    MPI_Comm comm            // 通信子
);
```

- Pour chaque indice `i` , MPI combine tous les `sendbuf[i]` des processus avec l'operateur `op` . / 对每个下标 `i` , MPI 都会把所有进程的 `sendbuf[i]` 用 `op` 合并。
- Le buffer `recvbuf` n'est significatif que sur le processus racine. / `recvbuf` 只对根进程真正有意义。
- Pour les flottants , de legeres differences numeriques peuvent apparaitre selon l'arbre de reduction utilise ; c'est normal. / 浮点归约的结果可能因归约树不同而有微小数值差异，这是正常的。

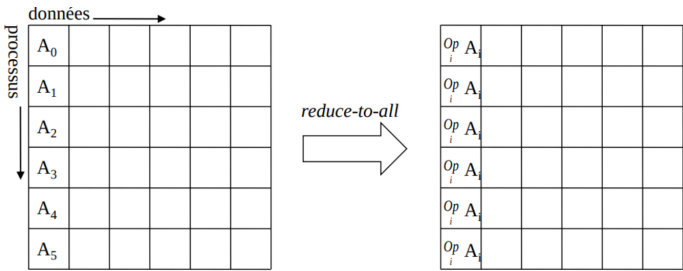
Operateurs predefinis frequents : / 常见预定义算子：

MPI_Op	MPI_MAX	MPI_MIN	MPI_SUM	MPI_PROD	MPI_MAXLOC	MPI_MINLOC
含义	最大值	最小值	求和	求积	最大值及其位置	最小值及其位置

```
int main(int argc, char *argv[]) {
    int rank, size, local_sum, global_sum;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    local_sum = rank; // 每个进程计算自己的局部和
    MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    // `MPI_Reduce` additionne les valeurs de `local_sum` de tous les processus et stocke le résultat dans `global_sum` du pi
    // `MPI_Reduce` 会把所有进程中的 `local_sum` 相加，并将结果存入进程 0 的 `global_sum` 中。
    if (rank == 0) {
        printf("Global sum: %d\n", global_sum);
    }
    MPI_Finalize();
    return 0;
}
```

4.7 MPI_Allreduce

MPI_Allreduce a la meme semantique de reduction que MPI_Reduce , mais le resultat final est ecrit dans le recvbuf de tous les processus. / MPI_Allreduce 与 MPI_Reduce 的归约语义完全相同，只是结果不是只留在根进程，而是写到所有进程的 recvbuf 。

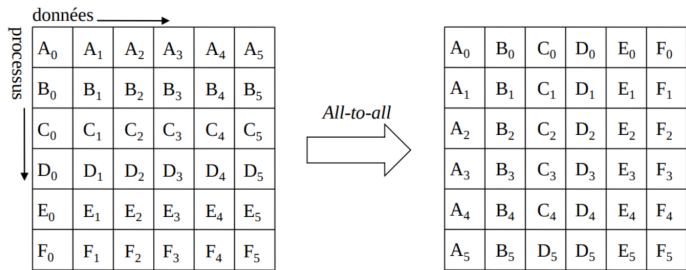


```
int MPI_Allreduce(
    void *sendbuf,           // 每个进程本地输入数据
    void *recvbuf,           // 每个进程本地输出缓冲区
    int count,               // 每个进程参与归约的元素个数
    MPI_Datatype datatype,   // 元素类型
    MPI_Op op,               // 归约算子
    MPI_Comm comm            // 通信子
);
```

```
int P, N = 100; // 计算 1 + 2 + ... + N 的并行版本
int moi, i, som_glob, som_local = 0;
MPI_Comm_rank(MPI_COMM_WORLD, &moi);
MPI_Comm_size(MPI_COMM_WORLD, &P);
/* hypothese : N est divisible par P */
for (i = 1 + moi * N / P; i <= (moi + 1) * N / P; i++) {som_local += i;}
MPI_Allreduce(&som_local, &som_glob, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
printf("1+...+%d = %d\n", N, som_glob); // 每个进程都有全局结果
```

4.8 MPI_Alltoall

MPI_Alltoall correspond a un echange total : chaque processus envoie un bloc a tous les autres processus et recoit aussi un bloc de chacun d'eux. / MPI_Alltoall 表示“每个进程都给每个进程发一块，也从每个进程收一块”。



```
int MPI_Alltoall(
    void *sendbuf,           // 本地发送数组, 里面按目标进程顺序存放各块
    int sendcount,           // 发给每个目标进程多少个 sendtype 元素
    MPI_Datatype sendtype,   // 发送元素类型
    void *recvbuf,           // 本地接收数组, 里面按源进程顺序存放各块
    int recvcount,           // 从每个源进程接收多少个 recvtype 元素
    MPI_Datatype recvtype,   // 接收元素类型
    MPI_Comm comm            // 通信子
);
```

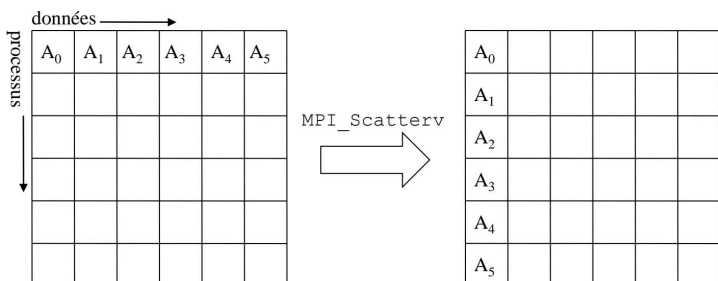
- Si la taille du communicateur est P , le `sendbuf` local doit contenir au moins $P * \text{sendcount}$ elements. / 若通信子大小为 P , 则本地 `sendbuf` 至少要能装 $P * \text{sendcount}$ 个元素。
- Le bloc destine au processus p commence localement a `sendbuf + p * sendcount`. / 发给进程 p 的块从本地 `sendbuf + p * sendcount` 开始。
- Le bloc recu du processus p est place localement a partir de `recvbuf + p * recvcount`. / 来自进程 p 的块写到本地 `recvbuf + p * recvcount` 开始。
- C'est une collective couteuse, car le volume total d'echange croit souvent comme P^2 . / 这是代价很高的 collective, 总交换量通常按 P^2 增长。
- On peut la voir comme une sorte de transposee d'une matrice de donnees repartie par processus. / 常见理解: 它像是在做“按进程划分的数据矩阵转置”。

4.9 变长版本: `*v`

Lorsque la taille des blocs depend de la source ou de la destination, il faut utiliser les versions variables portant le suffixe `v`. / 当每个目标/来源的块大小不一样时, 要用带后缀 `v` 的变长版本:

- `MPI_Scatterv`
- `MPI_Gatherv`
- `MPI_Allgatherv`
- `MPI_Alltoallv`

Prenons `MPI_Scatterv` comme exemple : / 以 `MPI_Scatterv` 为例:



```
int MPI_Scatterv(
    void *sendbuf,           // 根进程上的全局发送数组
    int *sendcounts,         // sendcounts[p] = 发给进程 p 的元素个数
    int *displs,             // displs[p] = 进程 p 这块数据在 sendbuf 中的起始位移
    MPI_Datatype sendtype,   // 发送元素类型
    void *recvbuf,           // 本地接收缓冲区
    int recvcount,           // 当前进程本地最多接收多少个元素
    MPI_Datatype recvtype,   // 接收元素类型
    int root,                // 根进程 rank
    MPI_Comm comm            // 通信子
);
```

- `sendcounts[p]` donne le nombre d'elements envoyes au processus p . / `sendcounts[p]` 决定每个进程分到的块大小。
- `displs[p]` donne la position de depart de ce bloc dans le tableau global. / `displs[p]` 决定该块在全局数组中的起始位置。
- Un schema classique consiste a diffuser ou scatter d'abord les longueurs locales, puis a allouer chaque `recvbuf` avec la bonne taille, et enfin a utiliser `MPI_Scatterv` pour envoyer les vraies donnees. / 常见套路: 先用一次 `MPI_Scatter` 把每个进程应接收的长度发出去, 再按该长度 `malloc`, 最后用 `MPI_Scatterv` 发真实数据。

Cours 5: MPI avance

5.1 通信器 communicator

- **Definition** : voir la section 2.1 sur le modele MPI. / 定义见 2.1 MPI 模型。
- **Pourquoi creer un nouveau communicateur** : pour regrouper des processus, isoler des espaces de communication et eviter d'ecrire beaucoup de conditions dans les communications collectives. / 为什么要建新通信器：把进程分组、隔离通信空间、避免在 collective 里写大量条件判断。

5.2 MPI_Comm_split

`MPI_Comm_split` separe un communicateur en plusieurs sous-communicateurs selon `color`, puis fixe l'ordre des rangs dans chaque sous-communicateur selon `key`. / `MPI_Comm_split` 按 `color` 把一个通信器分成若干子通信器，再按 `key` 决定每个子通信器里的 rank 顺序。

```
int MPI_Comm_split(
    MPI_Comm comm,          // 原通信器
    int color,              // 相同 color 的进程进同一个新通信器
    int key,                // 新通信器内部的排序键
    MPI_Comm *newcomm       // 输出：新通信器
);
```

- `color` decide seulement a quel groupe appartient un processus. / `color` 只决定“分到哪一组”。
- `key` decide seulement l'ordre des nouveaux rangs a l'interieur du groupe. / `key` 只决定“组内新 rank 顺序”。
- `color` n'a pas besoin d'etre continu ; des valeurs egales suffisent pour former un meme groupe. / `color` 不要求连续；只要相同就会被分到一组。
- Plus `key` est petit, plus le nouveau rang est place tot dans le sous-communicateur ; en cas d'egalite, l'ancien rang departage. / `key` 越小，新通信器里的 rank 越靠前；若 `key` 相同，用旧 rank 打破平局。
- Si un processus ne veut appartenir a aucun nouveau communicateur, il peut passer `MPI_UNDEFINED` et recoit alors `MPI_COMM_NULL`. / 若某进程不想加入任何新通信器，可传 `MPI_UNDEFINED`，此时返回 `MPI_COMM_NULL`。

Exemple minimal : separation entre rangs pairs et impairs. / 最短例子：偶数 / 奇数分组。

```
int rank, comm_pairs_impairs;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_split(MPI_COMM_WORLD, rank % 2, rank, &comm_pairs_impairs); // 按奇偶分组，组内保持原 rank 顺序
MPI_Bcast(a, m, MPI_FLOAT, 0, comm_pairs_impairs);                  // 每组各自做一次广播
MPI_Comm_free(&comm_pairs_impairs);                                  // 用完释放
```

5.3 其他通信器操作

```
int MPI_Comm_dup(MPI_Comm comm, MPI_Comm *newcomm); // 复制一个通信器，但创建新的通信上下文
int MPI_Comm_free(MPI_Comm *comm);                 // 释放通信器
```

- La duplication d'un communicateur avec `MPI_Comm_dup` est souvent utilisee dans les bibliotheques afin d'isoler leur contexte de communication de celui du reste de l'application. / `MPI_Comm_dup` 常用于库内部：复制出一个隔离的通信空间，避免和应用其他 MPI 组件互相干扰。
- Les operations de construction de communicateurs sont elles-memes en general des operations collectives. / 构造通信器的操作本身通常也是 collective。
- Un processus qui obtient `MPI_COMM_NULL` ne peut pas effectuer de communication ulterieure sur ce communicateur nul. / 一个进程若拿到的是 `MPI_COMM_NULL`，就不能在这个“空通信器”上做后续通信。

5.4 用户自定义归约：MPI_Op_create

`MPI_Op_create` permet de definir son propre operateur de reduction, puis de l'utiliser avec `MPI_Reduce` ou `MPI_Allreduce`. / `MPI_Op_create` 允许你定义自己的 reduction operator，然后把它交给 `MPI_Reduce` / `MPI_Allreduce` 使用。

```

typedef void MPI_User_function(
    void *invec,          // 新输入数据
    void *inoutvec,       // 累积结果；必须原地更新
    int *len,             // 元素个数
    MPI_Datatype *dtype   // 逻辑数据类型
);

int MPI_Op_create(
    MPI_User_function *function, // 用户自定义归约函数
    int commute,                // 是否可交换: 1=可交换, 0=不可交换
    MPI_Op *op                  // 输出: MPI 归约算子对象
);

int MPI_Op_free(MPI_Op *op);    // 释放用户定义的归约算子

```

- La fonction de reduction definie par l'utilisateur doit au minimum etre associative. / 你的归约函数至少必须满足结合律 **associative**。
- Si elle est aussi commutative, on met **commute = 1**, ce qui laisse davantage de possibilites d'optimisation a MPI. / 若同时满足交换律 **commutative**, 把 **commute** 设为 **1**, MPI 有更多优化空间。
- **invec** contient les nouvelles donnees, **inoutvec** joue le role d'accumulateur, et le resultat doit etre reecrit dans **inoutvec**. / **invec** 是新数据, **inoutvec** 是累积器, 结果必须写回 **inoutvec**。

```

void addem(void *invec, void *inoutvec, int *len, MPI_Datatype *dtype) {
    int *in = invec, *inout = inoutvec;
    for (int i = 0; i < *len; i++) inout[i] += in[i]; // 原地累加
}

MPI_Op op;
MPI_Op_create(addem, 1, &op);
MPI_Reduce(sendbuf, recvbuf, count, MPI_INT, op, root, MPI_COMM_WORLD);
MPI_Op_free(&op);

```

5.5 派生类型 **Types dérivés**

- 定义: En MPI, un type dérivé est un type de données construit à partir de types MPI de base afin de décrire une organisation particulière des données en mémoire. Il permet de communiquer en une seule opération des données non contiguës ou structurées. / 在 MPI 中, 派生类型是由基本 MPI 类型构造出来的数据类型, 用来描述内存中某种特定的数据布局。它可以让我们一次通信发送非连续的数据或结构化数据。
- Le probleme resolu par les types derives est que la disposition memoire n'est pas toujours un simple bloc contigu d'elements de base. / 派生类型解决的问题: 内存布局不是“简单连续的一段基础类型”。
- Usages typiques : envoyer une colonne de matrice, des donnees a stride, une structure ou des blocs non contigus. / 典型用途: 发送矩阵列、步长访问数据、结构体、非连续块。
- Tout type derive doit etre cree, puis valide avec **MPI_Type_commit**, puis libere avec **MPI_Type_free** apres usage. / 任何派生类型创建后, 都必须先 **MPI_Type_commit**, 用完再 **MPI_Type_free**。

Exemple 1 : / Example 1 :

```
#include <mpi.h>
#include <stdio.h>
#include <stddef.h>
typedef struct {
    int id;
    double value;
} Data;
int main(int argc, char *argv[]) {
    int rank;
    MPI_Datatype mpi_data_type;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    Data d;
    int block_lengths[2] = {1, 1};
    MPI_Aint displacements[2];
    MPI_Datatype types[2] = {MPI_INT, MPI_DOUBLE};
    displacements[0] = offsetof(Data, id);
    displacements[1] = offsetof(Data, value);
    MPI_Type_create_struct(2, block_lengths, displacements, types, &mpi_data_type);
    MPI_Type_commit(&mpi_data_type); // 提交后才能用于通信
    if (rank == 0) {
        d.id = 1;
        d.value = 3.14;
        MPI_Send(&d, 1, mpi_data_type, 1, 0, MPI_COMM_WORLD);
    } else if (rank == 1) {
        MPI_Recv(&d, 1, mpi_data_type, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("id = %d, value = %f\n", d.id, d.value);
    }
    MPI_Type_free(&mpi_data_type); // 用完释放
    MPI_Finalize();
    return 0;
}
```

Ici, on crée un type dérivé MPI correspondant à la structure Data. Cela permet d'envoyer en une seule communication les deux champs id et value, même si la structure est composée de plusieurs types de base. / 这里我们创建了一个和结构体 Data 对应的 MPI 派生类型。这样就可以在一次通信中发送 id 和 value 两个字段，虽然它们属于不同的基本类型。

Exemple 2 : / Example 2 :

Si on veut envoyer une colonne d'une matrice stockée en mémoire ligne par ligne, on peut utiliser un type dérivé pour décrire cette colonne, car ses éléments ne sont pas contigus en mémoire. / 如果要发送一个按行存储矩阵中的某一列，可以用派生类型描述这一列，因为这一列在内存中不是连续存放的。

5.6 常见派生类型构造器

构造器	适用场景	步长单位
MPI_Type_contiguous	一段连续重复的旧类型	无步长
MPI_Type_vector	固定块长 + 固定间隔的重复块	元素个数
MPI_Type_create_hvector	固定块长 + 固定间隔的重复块	字节数
MPI_Type_indexed	可变块长 + 可变位移	元素个数
MPI_Type_create_hindexed	可变块长 + 可变位移	字节数
MPI_Type_create_struct	结构体 / 异构字段	字节位移

Differences essentielles : / 关键区别：

- vector utilise un stride exprime en nombre d'elements de l'ancien type. / vector 的 stride 单位是“旧类型元素个数”。
- hvector utilise un stride exprime en octets. / hvector 的 stride 单位是“字节数”。
- struct et hindexed exigent des déplacements precis afin de prendre correctement en compte l'alignement et le padding. / struct / hindexed 这类接口必须精确给出位移，以正确处理对齐和 padding。

5.7 三个最常用签名

```
int MPI_Type_contiguous(
    int count,           // 连续重复多少个 oldtype
    MPI_Datatype oldtype, // 原类型
    MPI_Datatype *newtype // 输出：新类型
);

int MPI_Type_vector(
    int count,           // 块数
    int blocklength,     // 每块长度
    int stride,          // 相邻块起点间隔；单位是 oldtype 个数
    MPI_Datatype oldtype, // 原类型
    MPI_Datatype *newtype // 输出：新类型
);

int MPI_Type_create_hvector(
    int count,           // 块数
    int blocklength,     // 每块长度
    MPI_Aint stride,     // 相邻块起点间隔；单位是字节
    MPI_Datatype oldtype, // 原类型
    MPI_Datatype *newtype // 输出：新类型
);
```

Schema minimal : / 最短套路：

```
MPI_Datatype type_colonne;
MPI_Type_vector(nb_lignes, 1, nb_colonnes, MPI_DOUBLE, &type_colonne); // C 里矩阵列通常是跨步的
MPI_Type_commit(&type_colonne);
MPI_Send(&a[0][0], 1, type_colonne, 1, tag, MPI_COMM_WORLD);
MPI_Type_free(&type_colonne);
```

- Dans un tableau C stocke en ordre ligne, une colonne complete n'est generalement pas contigue en memoire ; on utilise donc souvent `MPI_Type_vector` . / 在 C 的行主序数组里，“一整列”通常不是连续内存，所以常用 `MPI_Type_vector` 。
- Dans un tableau Fortran stocke en ordre colonne, une colonne peut etre naturellement contigue ; la construction du type peut alors etre differente. / 在 Fortran 的列主序数组里，列可能天然连续，这时构造方式会不同。